

Lecture 12



PDF

Lecture 13



PDF

In my deep learning class, I learned about GNNs and how they can be very applicable in network settings. I have attached my GNNs note to this chat.

Let us first start by giving an introduction into GNNs an...



Show thinking

It is great to have you here! Graph Neural Networks (GNNs) are indeed a powerful tool for modeling relational data. Based on your lecture notes, we can look at GNNs not just as a new tool, but as a logical next step from concepts you likely already know, like Convolutional Neural Networks (CNNs).

Here is an introduction to the core concepts found in your notes:

1. Generalizing CNNs to Graphs

Your notes introduce GNNs by comparing them to CNNs. You can think of an image as a very specific type of graph: a regular grid where every "node" (pixel) has a fixed number of neighbors (top, bottom, left, right).

- **The Difference:** In a general graph (like a social network), nodes can have any number of neighbors, and there is no fixed order (no "top-left" neighbor).

Commented [1]: I liked how Gemini followed a same approach as the lecture notes by comparing CNNs to GNNs.

- **The Similarity:** Despite these differences, GNNs borrow the key idea of **weight sharing** from CNNs. We use the same learnable parameters (matrices) to process every node, just as a CNN uses the same filter across an entire image.  

2. Message Passing and Aggregation

Since we cannot rely on a fixed grid structure, GNNs use a mechanism often called "message passing."

- **Gathering Info:** A node gathers (aggregates) information from its neighbors. 
- **Permutation Invariance:** Because the order of neighbors does not matter, the function we use to combine this information must be "permutation invariant." Your notes suggest operations like **Sum**, **Max**, **Mean**, or **Variance** to combine neighbor signals. 

3. GNN Tasks

Your notes outline three main levels at which GNNs operate:  

- **Graph Classification:** Classifying the whole graph (e.g., is this molecule toxic?).
- **Node Classification:** Predicting a label for a specific node (e.g., is this user a bot?).
- **Edge Prediction:** Predicting if a connection exists between two nodes.

To help you get a better grasp of these mechanics, which area would you like to explore first?

1. **The "Convolution" Analogy:** We can look closer at how the math of a 3x3 convolution on an image translates to a graph node summing up messages from its neighbors.
2. **Aggregation Functions:** We can discuss why we might choose "Max" over "Sum" or "Mean" when combining neighbor information, and how that decision changes what the network learns.

3. **Handling "Pooling"**: In CNNs, we make images smaller (pooling) to see the big picture. We can explore how your notes suggest doing this on a graph, where there is no easy way to just "downsample."

I'm really interesting in understanding how GNNs are applied to social networks. Based on this, please choose which area I should explore first.

◆ Show thinking ▼

That is a perfect context to have in mind! Social networks are actually the textbook example for why we need Graph Neural Networks (GNNs) in the first place.

Given that interest, I recommend we start with **1. The "Convolution" Analogy**.

Here is why: deeply understanding the difference between "filtering an image" and "filtering a social network" will give you the intuition for everything else (like how we handle a user with 5 friends vs. a user with 5,000 friends).

To kick things off, let's look at the structure of the data.

- **In an Image (CNN)**: If you pick a specific pixel, you always know exactly where its neighbors are (top, bottom, left, right).
- **In a Social Network (GNN)**: You are a "node" and your friends are your "neighbors." 🗺️

Commented [2]: I liked how Gemini catered my study plan towards the application of social networks and suggesting what order to learn the GNN concepts.

Thinking about a typical social network, what is the biggest structural difference between a pixel's neighbors and a person's neighbors?

We don't know exactly where a person's neighbors are. On the other hand, we know exactly where a pixel's neighbors are.

◆ Show thinking ▼

Spot on. That lack of "spatial order" is the key constraint.

In an image (CNN), because we know exactly where "top-left" is, we can learn a specific weight just for that position. But in a social network, your friends are just a "set" or a "bag" of connections. If we re-ordered your friends list, it shouldn't change the output of the neural network.

This property is called **permutation invariance**.

To handle this, GNNs usually process every neighbor using the *same* learnable weights, and then squash all that information into a single "message" using a summary operation.

Imagine you are a node in a social network, and you have information about the **age** of all your adjacent friends (neighbors).

What simple mathematical operations could you use to combine a list of 5 ages (or 500 ages) into a single number, such that the result doesn't depend on the order of the list?

Commented [3]: This was a great introduction into permutation invariance. Using social networks as the motivation behind this property really helped me understand this concept.

We could use an average.

◆ Show thinking ▼

Exactly! Taking the average is a perfect example because $\frac{A+B+C}{3}$ is the same as $\frac{C+A+B}{3}$. The order doesn't matter, which satisfies our need for **permutation invariance**.

Your notes (Lecture 12) actually list "Average" alongside other options like Sum, Max, Variance, and even "Count Unique."

The "Graph Convolution"

This leads us to the core formula for a GNN layer. In a CNN, we slide a filter over the image. In a GNN, the "filter" is this process of gathering and averaging.

Mathematically, for a node h_i , the update often looks something like this:

$$h_i^{new} = \sigma(W \cdot \text{AVERAGE}(\text{neighbors of } i))$$

(Where W is that shared weight matrix we talked about, and σ is a non-linearity like ReLU).

The Missing Piece

Now, imagine we are using this to predict if **you** (Node i) will like a new movie. We successfully grab the average taste of all your friends (neighbors).

If we **only** use that average to update your node's state, what crucial piece of data are we completely ignoring?



We are completely ignoring our own current state.

◆ Show thinking ▼

That is exactly right! 🎯

If we only looked at the average of your neighbors, the model would treat you purely as a reflection of your friends, completely ignoring your own unique characteristics (like your age, your own past movie ratings, or your bio).

The Fix: The Self-Loop

To solve this, GNNs explicitly include the node's current state in the update. You can think of this as adding a "self-loop" to the graph—an edge that connects you to yourself so your own information flows into the next layer.

So now we have **two** distinct sources of information coming into Node i :

1. **Neighbor Info:** The aggregated "message" from friends (e.g., the average).
2. **Self Info:** The node's current representation (h_i).

Combining the Signals

In your lecture notes (Lecture 12, Page 7), this is formalized as a function f that takes both pieces:

$$h_i^{new} = f(h_i, \text{Aggregated_Neighbors})$$

Commented [4]: This was a great segue into taking about the update function in GNNs. I wish the notation of the function f could have been a bit more rigorous and descriptive though.

In standard neural networks, if you have two input vectors (your own features and the neighbor features) and you want to process them, what are the two most common ways to mathematically "combine" them before multiplying by weights?

Hmm...I'm not too sure. Can you give me a small hint?

✦ Show thinking ▾

No problem at all! It is a bit abstract until you see it written out.

Think about it in terms of vectors (lists of numbers).

If you have a vector representing **you** (e.g., `[MyAge, MyHeight]`) and a vector representing the **average of your friends** (e.g., `[AvgFriendAge, AvgFriendHeight]`), how can we feed *both* into the next layer?

There are two very common algebraic tricks we use in deep learning:

1. **Stacking:** You can "glue" the two vectors together side-by-side to make one longer vector. (This is often called **concatenation**).
2. **Blending:** You can transform them separately and then **add** them up.

Does **concatenation** (sticking them together) sound like a reasonable way to make sure the model sees *both* your info and your friends' info?

👍 🗨 📄 ⋮

Yeah I was thinking concatenaton. Thanks for the hint.

✦ Show thinking ▾

Concatenation is a solid choice! 🧠 It is the most straightforward way to ensure the model has access to distinct "self" and "neighbor" information without blurring them together.

So, visually, our input to the update step now looks like one long vector: `[ My_Features | Aggregated_Neighbor_Features ]`

Now we need to process this. In your lecture notes (Lecture 12), it mentions using an **MLP** (Multi-Layer Perceptron) to do this combination.

In a standard neural network (or MLP), once you have an input vector like this, what mathematical object do you multiply it by to mix the features and transform them into the output?

👍 🗨 📄 ⋮

Some weight matrix.



Show thinking ▾

Exactly. You multiply by a learnable **Weight Matrix** (W). This matrix is what allows the network to learn *how* to mix your personal features with your social circle's features to make a prediction.

So, putting it all together, a single layer of a GNN often looks like this:

$$h_i^{new} = \sigma(W \cdot [h_i \parallel \text{AGGREGATE}(\text{neighbors})])$$

(Where $\parallel$ means concatenation).

This covers the "Convolution" analogy! You've got the core mechanics down.

2. Aggregation Functions: Sum vs. Mean vs. Max

Now let's move to the second area you were interested in: **Aggregation Functions**.

We used **Average** (Mean) in our previous example because it's intuitive. But in social networks, the choice between **Sum**, **Mean**, and **Max** can completely change what the model learns.

Let's try a thought experiment related to social network security. Imagine we are trying to detect **bot accounts** based on how many "fake" accounts they are connected to.

- **User A** has 10 friends. 5 are known bots (50%).
- **User B** has 100 friends. 50 are known bots (50%).

If our aggregation function is the **Average** (Mean), the network sees the input "0.5" for both users.

Intuitively, do you think a fraud detection system should treat User A and User B exactly the same, or is one situation potentially more significant than the other?

Commented [5]: Overall, I was pretty impressed with how Gemini incorporated social networks into explaining the convolution strategy in GNNs.

It should treat User B's situation more significantly.

◆ Show thinking ▼

You are absolutely right. 🛡️ In a security context, a network of 50 bots is a much larger threat vector than a network of 5, even if the *ratio* is the same.

The **Average** (Mean) function fails us here because it "normalizes" the data—it divides by the number of neighbors, effectively throwing away information about the **size** of the user's social circle (the node degree).

To capture that magnitude—so that User B generates a much stronger "bot alert" signal than User A—which of the other aggregation functions we listed (Sum or Max) would be the better choice?

👍 🗨️ 📄 ⋮

Max?

◆ Show thinking ▼

That is a very logical guess, but in this specific case, **Max** would actually fail to tell the difference!



If we treat a bot as a "1" and a real person as a "0", let's see what the **Max** function sees:

- **User A (5 bots):** The maximum value in their friend list is **1** (because they have at least one bot).
- **User B (50 bots):** The maximum value is *also* **1**.

Max detects the *presence* of a signal, but it completely ignores the *quantity*.

To catch the "army" of bots (User B), we want to count up the total threat magnitude. **Sum** does this perfectly:

- **User A:** $1 + 1 + 1 + 1 + 1 = 5$
- **User B:** $1 + \dots + 1 = 50$

Now the network clearly sees that User B is the bigger anomaly.

However, **Max** is still extremely useful in other settings.

Imagine we are using a GNN to classify a chemical molecule. If the molecule contains **even one** highly toxic atom, the entire molecule is considered poisonous.

In that "single toxic atom" scenario, why would **Max** be a much safer choice than **Average**?

👍 💬 📄 ⋮

Because if max guarantees we determine the molecule is poisonous if there a toxic atom, while average doesn't



Show thinking ▾

That is exactly it. `Max` is a "detector"—it flashes a warning if *any* signal is present, which is critical for things like toxicity or specific keywords. `Average` washes that signal out if the rest of the nodes are "safe."

You have now mastered the two biggest building blocks:

1. **Convolution:** Passing messages from neighbors (and yourself).
2. **Aggregation:** Deciding how to combine those messages (Sum vs. Max vs. Mean).

3. Handling "Pooling" (Graph Coarsening)

Let's hit the last major concept you asked about: **Pooling**.

In a standard image network (CNN), we make the image smaller every few layers (e.g., turning a 64×64 image into a 32×32 image). This helps the model "zoom out" and see global patterns rather than just pixel details.

In a social network, we often want to do the same thing: zoom out from "individual users" to "communities."

But here is the catch: In an image, we just draw little 2×2 boxes on the grid and squash them. We can't do that on a graph because there is no fixed grid!

If you were trying to simplify a massive social network by merging users into groups (super-nodes), what criteria would you use to decide which users belong in the same group?

👍 🗨️ 📄 ⋮

Commented [6]: Gemini didn't dive as deep into the aggregation strategy as I wish it could have.

We could use maybe check connected components in the graph?

◆ Show thinking ▼

That is a very logical first thought! Connected components definitely group nodes together.

However, there is a catch with social networks: they often have one massive "giant component" where almost everyone is linked to everyone eventually (the "six degrees of separation" idea). If we used connected components, we might just get one huge super-node containing 99% of the graph!

Instead of looking for nodes that are totally disconnected from the rest, we want to find groups that are *densely* connected internally.

Think about a "community" or a "clique" of friends. Compared to their connections with the rest of the world, are their connections with each other usually **stronger/more frequent** or **weaker/less frequent**?

👍 🗨 📄 ⋮

Stronger and more frequent

Commented [7]: The question about determining what criteria to use to decide how to form groups from users was a bit vague. I think Gemini could have asked a more specific question.



Show thinking ▾

Exactly! 🧠 A "community" is defined by having dense, strong connections internally, even if the connections to the outside world are sparse.

Graph Pooling: The "Cluster" Strategy

This gives us our strategy for pooling on graphs (often called **Graph Coarsening**):

1. **Find Clusters:** Identify these tightly knit groups of nodes (communities).
2. **Merge Them:** Squash each group into a single "super-node."
3. **Update Edges:** If two original communities had connections between them, we draw an edge between their new super-nodes.

By doing this, we can turn a graph of N **users** into a much smaller graph of K **communities** ($K < N$).

The Assignment Matrix

To do this mathematically, the network needs to learn which user belongs to which community. We represent this with a learned **Assignment Matrix** (often denoted as S).

If we have N original nodes and we want to map them to K clusters, what would the **dimensions** (rows $\times$ columns) of this matrix S be?

👍 🗨 📄 ⋮

It should be K by N.

That is a very natural way to think about it! In linear algebra, if we want to transform a vector of size N into a vector of size K , we typically left-multiply by a $K \times N$ matrix.

However, in the GNN world (and most deep learning libraries), we usually flip this convention and define the assignment matrix S as $N \times K$.

Here is the intuition for why we organize it that way:

- **Rows (N):** Each row corresponds to one of the original nodes.
- **Columns (K):** Each column corresponds to one of the new "clusters" (super-nodes).

This way, if you look at **Row 1**, you can read it as a probability distribution: "Node 1 belongs 90% to Cluster A and 10% to Cluster B."

Computing the New Features

Now, let's use this matrix to actually "pool" the data.

We have:

1. H : The node feature matrix of size $N \times F$ (N nodes, F features each).
2. S : The assignment matrix of size $N \times K$.

We want to end up with a new matrix H_{new} of size $K \times F$ (features for our K new super-nodes).

Using standard matrix multiplication, how would you multiply S and H together to get that $K \times F$ result?

👍 🗨 📄 ⋮

Commented [8]: I'm not really sure why Gemini said the answer was wrong, since most mathematically applications use left-multiplication.

Hey, I appreciate the help with understanding GNNs through social networks. Since the conversation is getting a bit too long, can you finish off by giving me a concise summary of the connection between GNNs and social networks?

◆ Show thinking ▼

It has been a pleasure exploring this with you! Breaking down complex topics like GNNs using real-world analogies is exactly how we build intuition.

Here is a concise summary of how Graph Neural Networks map perfectly onto Social Networks:

- **The Structure:** Unlike images (which are rigid grids), social networks are irregular. **Users are nodes**, and **friendships are edges**. Because you can have any number of friends in any order, GNNs rely on **permutation invariance**—meaning the math doesn't change just because we shuffled your friends list. ⓘ
- **The "Social" Convolution (Message Passing):** Instead of sliding a window like a CNN, a GNN node learns by **aggregating information** from its neighbors. ⓘ
 - **Aggregation:** We use functions like **Sum** (to count total influence, like a bot army) or **Average** (to see typical behavior) to summarize friend data. ⓘ
 - **Update:** The node combines this "neighbor summary" with its own personal features (the **self-loop**) to update its state. ⓘ
- **Community Detection (Pooling):** To understand the network at a macro level, we use **pooling**. We identify tightly knit groups of friends (cliques) and merge them into "super-nodes" to analyze communities rather than just individuals.

Commented [9]: From here, I thought that Gemini was hallucinating a bit. At this point, it felt like too long of conversation about explaining GNNs through social networks. Overall, Gemini did a great job of explaining GNNs in the lens of social networks, but I feel it was going a little bit all over the place towards the end.